

# Tableau Extensions 101

By Mohit Kumar – Aug 02, 2026

---

Tableau is one of the most powerful Business Intelligence (BI) and data visualization platforms available today. While Tableau offers an extensive set of built-in features, there are times when organizations need capabilities that go beyond what is available out of the box. That's where Tableau Extensions come into play.

Tableau Extensions allow developers and businesses to integrate external applications, custom functionality, and third-party tools directly into Tableau dashboards, creating richer and more interactive analytics experiences.

The motivation for writing this blog comes from Eric Summers, who has done a remarkable job of making Tableau Extensions easier to build. If you're unfamiliar with his work on Extensions, I highly recommend checking out his profile.

For a long time, I've been using the extensions created by Tristan Guillevin. Like many other Tableau users, I've used his extensions to build great visualizations on Tableau Public. Recently, I decided it was time to build one myself. But before getting started, it's important to understand what a Tableau Extension is. So, here we go...

## What Are Tableau Extensions?

A Tableau Extension is essentially a web application that can be embedded inside a Tableau dashboard. It is made using things like HTML and CSS and JavaScript. The Tableau Extension talks to Tableau using the Extensions API.

A Tableau Extension is different from a web page that you might put in a dashboard. A Tableau Extension can talk to the dashboard. It can read data from worksheets, respond to user selections, apply filters, access parameters, and even communicate with external applications. For the end user, everything feels like part of the same dashboard, even though the extension may be communicating with multiple external applications behind the scenes. This opens up a wide range of possibilities that go beyond traditional data visualization.

## Components of Tableau Extensions

A basic Tableau Extension usually consists of the following files:

- Extension Manifest File (.trex)
- HTML File
- JavaScript File
- CSS File

Let's understand what each file is.

## Extension Manifest File (.trex)

The .trex file is the most important file in a Tableau Extension. It tells Tableau how to load and display the extension. This file contains information such as:

- Extension name
- Extension description
- Defines the encodings available for a Viz Extension
- Location of the web application
- Required permissions
- Whether the extension is safe or network-enabled

When you add an extension to a Tableau dashboard, Tableau reads this file first to understand what needs to be loaded.

A simple .trex file looks something like this:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest manifest-version="0.1">
  <dashboard-extension id="com.example.myextension">
    <default-locale>en_US</default-locale>
    <name>My Tableau Extension</name>
    <description>
      A sample Tableau Extension
    </description>
    <author>
      <name>Your Name</name>
    </author>
    <min-api-version>1.0</min-api-version>
    <source-location>
      <url>http://localhost:8080/index.html</url>
    </source-location>
  </dashboard-extension>
</manifest>
```

The most important part here is the source-location. It tells Tableau where the extension application is hosted.

## HTML File

The HTML file is the entry point of your extension. Similar to any web application, it defines the structure of the page that will appear inside your Tableau dashboard. Basically, it loads API and creates display area.

A typical extension starts by loading JavaScript libraries and creating the basic page structure.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Tableau Extension</title>
</head>
<body>
  <h2>My First Tableau Extension</h2>
  <button id="applyFilter">
    Apply Filter
  </button>
  <script src="script.js"></script>
</body>
</html>
```

The HTML file itself does not communicate with Tableau. That responsibility belongs to JavaScript.

## JavaScript File

JavaScript is where the actual interaction with Tableau happens.

The Tableau Extensions API is a JavaScript library that allows your extension to communicate with the dashboard. Without the Extensions API, your extension would just be a normal web page sitting inside a dashboard. It would not know anything about the Tableau workbook, worksheets, filters, or user interactions. The API gives your extension the ability to "talk" to Tableau. Using JavaScript, you can:

- Connect to the current Tableau dashboard
- Access worksheets
- Read selected data points
- Apply filters
- Change parameters
- Trigger dashboard actions
- Communicate with external applications

The first step is usually initializing the Tableau Extension. Example:

```
tableau.extensions.initializeAsync()

.then(function () {
  console.log("Extension initialized");
});
```

Once initialized, the extension can access the Tableau environment.

For example, getting the worksheet content using **tableau.extensions.worksheetContent.worksheet**

From here, you can interact with worksheets and dashboard objects. You could add the JavaScript code to initialize and call Extensions API functions directly in the HTML page. However, in most cases you want to keep this code in a separate file.

Well, you can and should create different JavaScript files. Keeping the Tableau Extensions API JavaScript file separate from the JavaScript files you create for your extension improves readability and makes the code easier to understand. However, you do not need two separate JavaScript files to build a Tableau Extension.

## CSS File

CSS controls the look and feel of the extension. Since extensions are web applications, you can style them just like any other website. CSS can be used to customize Fonts, Colors, Layout, Buttons, Tables, Forms, etc.

Example:

```
button {  
  background-color: #1f77b4;  
  color: white;  
  padding: 10px;  
  border-radius: 5px;  
}
```

A well-designed interface is important because the extension becomes part of the user's dashboard experience.

## How Does a Tableau Extension Actually Work?

The working process of a Tableau Extension is quite simple.

1. **Tableau Loads the Extension:** When a user places an extension on a dashboard, Tableau reads the .trex file. The manifest file tells Tableau where the extension application is located.
2. **The Web Application Opens:** Tableau loads the HTML page specified in the .trex file. The HTML page then loads the required JavaScript and CSS files.
3. **Extension Connects with Tableau:** The JavaScript code initializes the Tableau Extensions API. At this point, the extension can communicate with the dashboard.

4. User Interacts with the Extension: When a user clicks a button, selects an option, or performs an action, JavaScript handles the event. The extension can then:
  - Read information from Tableau
  - Send commands back to Tableau
  - Update dashboard filters
  - Retrieve external information
5. Tableau Updates the Dashboard: Based on the action performed, Tableau responds and updates the dashboard accordingly.

In simple terms:

- The .trex file tells Tableau what to load and where to find the extension. It holds the web address.
- The HTML file creates the basic structure of the extension.
- The CSS file controls how the extension looks and feels.
- The JavaScript file adds the logic and allows the extension to communicate with Tableau.
- A backend application (if needed) handles advanced tasks like connecting to databases, APIs, or other systems.

## Hosting Your Tableau Extension

So far, we have covered the main components of a Tableau Extension. However, one important piece is still left... the source-location in the .trex file.

The source-location tells Tableau where your extension application is hosted. Tableau does not directly read files from your local folder. The extension files need to be served through a web server.

For a production-ready Tableau Extension, your files should be hosted on a proper server with a secure HTTPS connection. This allows Tableau to access and load your extension reliably.

However, if you are just building and testing your extension locally, you can use localhost. A simple way to do this is by using Python's built-in HTTP server.

Open a terminal in the folder where your extension files are stored and run the following command:

```
python -m http.server 8000
```

This will start a local web server and host your files at: `http://localhost:8000`

You can then use this URL in the source-location section of your .trex file while testing your extension.

For example:

```
<source-location>

  <url>http://localhost:8000/index.html</url>

</source-location>
```

This setup is perfect for development and testing. Once your extension is ready to be shared with users, you should host it on a proper web server.

## Extension Types

Tableau Extensions are mainly divided into two types:

- Dashboard Extensions
- Viz Extensions

Although both extensions are built using the Tableau Extensions API, they serve different purposes and work at different levels within Tableau.

### Dashboard Extensions

Dashboard Extensions are custom web applications that you can drag and drop directly onto a Tableau dashboard.

They are mainly used when you want to add functionality beyond Tableau's native dashboard capabilities. A Dashboard Extension can interact with the dashboard and help you create custom experiences such as:

- Adding custom controls
- Connecting Tableau with external applications
- Enabling write-back functionality
- Automating business workflows

Dashboard Extensions primarily interact with the dashboard using `dashboardContent` from the Tableau Extensions API.

In simple terms, Dashboard Extensions enhance the overall dashboard experience.

### Viz Extensions

Viz Extensions work differently from Dashboard Extensions. Instead of being added to a dashboard, they are added while building a visualization through the Marks card in a worksheet.

Viz Extensions allow developers to create custom visualization types that work directly within the Tableau authoring experience.

For example, instead of choosing a standard bar chart or line chart, users can select a custom visualization created through a Viz Extension.

Viz Extensions use `worksheetContent` from the Tableau Extensions API, which gives them access to the data and contents of a specific worksheet.

## Building your first viz extension

That was a lot of theory! Now, let's build our first viz extension. We will start with the simplest one, an extension that displays a simple "Hello" or "Welcome" message when it loads.

Since this extension only displays a simple welcome message, we can build it using just two files: a `trex` file and an HTML file. The `.trex` file tells Tableau where the HTML file is located and which file needs to be loaded when the extension starts.

1. Let's start by creating a folder named `Welcome Extension`.
2. Inside this folder, create a manifest file named `Welcome.trex`. This file will contain the basic configuration details required by Tableau to load our extension.

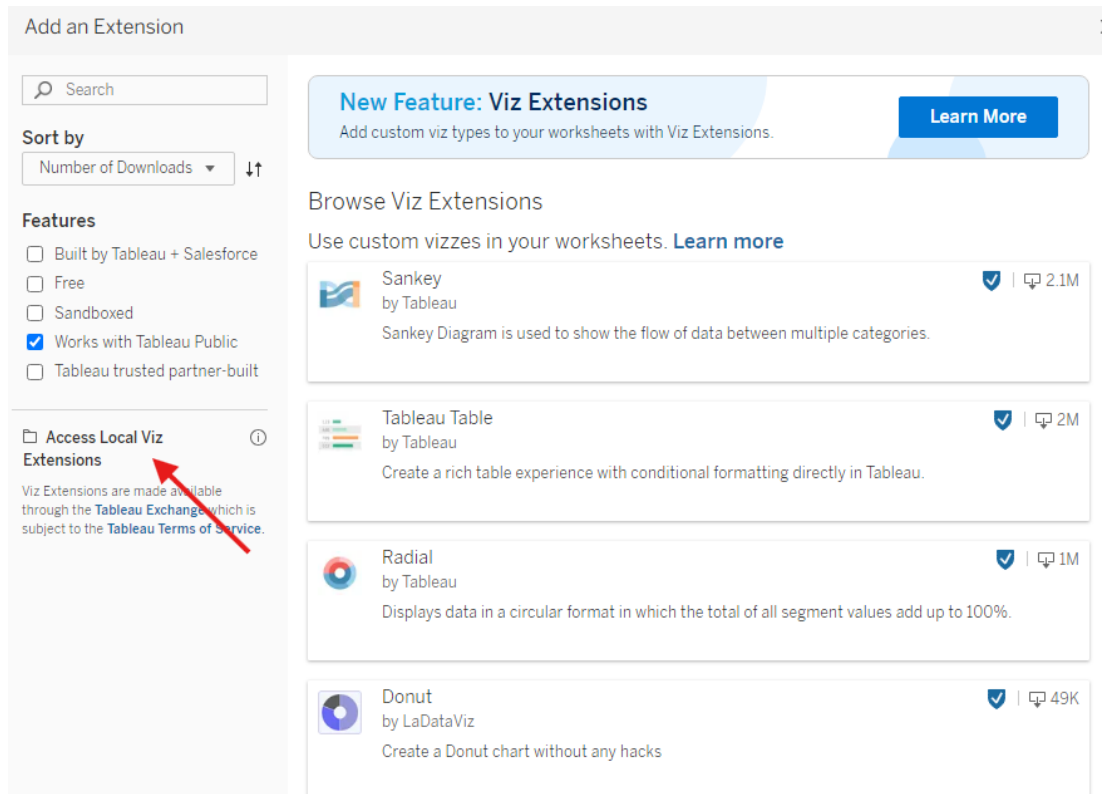
```
<?xml version="1.0" encoding="utf-8"?>
<manifest manifest-version="0.1"
xmlns="http://www.tableau.com/xml/extension_manifest">
  <worksheet-extension id="com.tableau.extension.first.welcome" extension-
version="0.1.0">
    <default-locale>en_US</default-locale>
    <name resource-id="name"/>
    <description>Welcome Extension</description>
    <author name="Mohit" email="email@tableau.com" organization="tableau"
website="https://www.tableau.com"/>
    <min-api-version>1.1</min-api-version>
    <source-location>
      <url>http://localhost:8081/src/Welcome.html</url>
    </source-location>
    <icon/>
  </worksheet-extension>
  <resources>
    <resource id="name">
      <text locale="en_US">Welcome Extension</text>
    </resource>
  </resources>
</manifest>
```

3. Now, create a subfolder named `src` inside the `Welcome Extension` folder, and create a file named `Welcome.html` inside it.

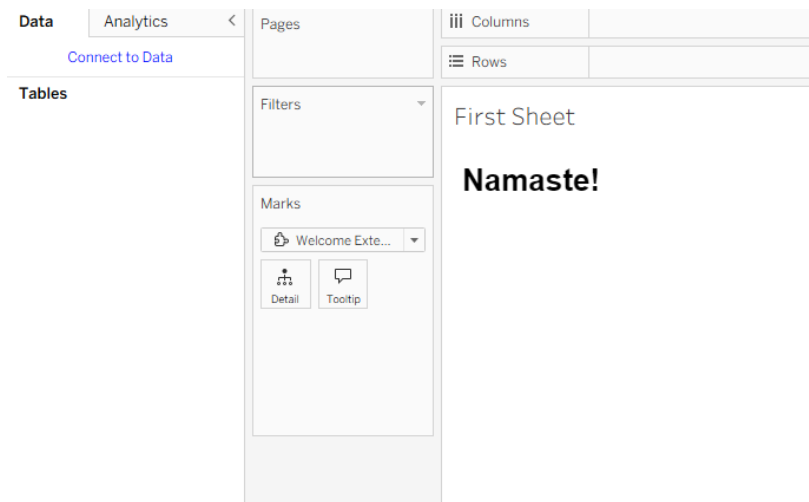
```
<!DOCTYPE html>
<html>
  <head>
    <title>Welcome Extensions</title>
  </head>
```

```
<body>
  <h1>Namaste! :)</h1>
</div>
</body>
</html>
```

- Now that we have both files in place, let's start our server. Open Command Prompt and change the directory to the **Welcome Extension** folder.
- Enter command **python -m http.server 8081** to start the server.
- Next, open Tableau and go to a worksheet. Rename the sheet to First Sheet.
- In the Marks Card, select **Add Extension**, then choose **Access Local Viz Extensions**.



Select your **WelcomeExtension.trex** file. You should see below output:



Now, to make things more interesting, let's display the name of the worksheet along with the welcome message. To achieve this, we will use the Tableau Extensions API. Let's make the required modifications.

8. First, download the Tableau Extensions API JavaScript file from the location below:  
<https://github.com/tableau/extensions-api/blob/main/lib/tableau.extensions.1.latest.js>
9. Create a subfolder named `lib` and place the downloaded `tableau.extensions.1.latest.js` file inside it.
10. Next, create a file named `Welcome.js` in the `src` folder.

```
'use strict';
(function () {
  tableau.extensions.initializeAsync()
    .then(() => {
      const worksheet =
        tableau.extensions.worksheetContent.worksheet;
      document.getElementById("content").innerHTML =
        `Your worksheet name is ${worksheet.name}.`;
    })
    .catch((err) => {
      console.log("Initialization Error:", err);
    });
})();
```

11. Update your `Welcome.html` file with the required changes.

```
<!DOCTYPE html>
```

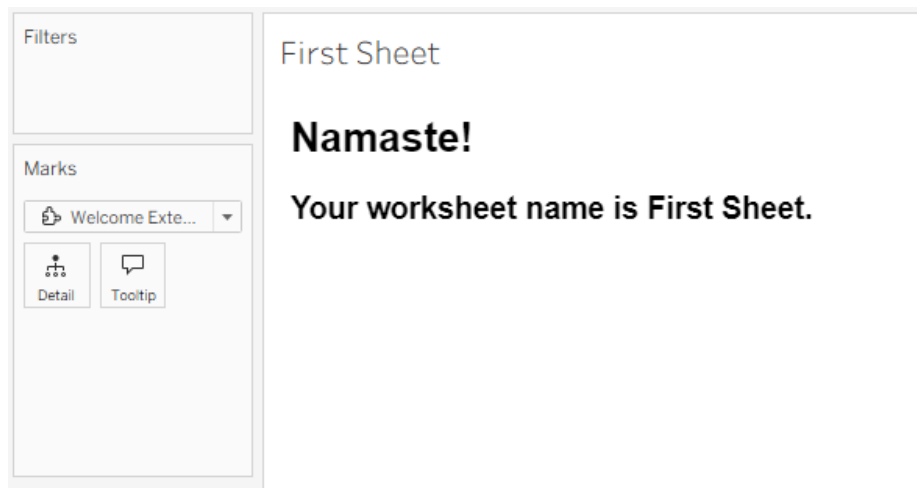
```

<html>
<head>
  <title>Welcome Extension</title>
  <!-- Tableau Extensions API Library -->
  <script src="../../lib/tableau.extensions.1.latest.js"></script>

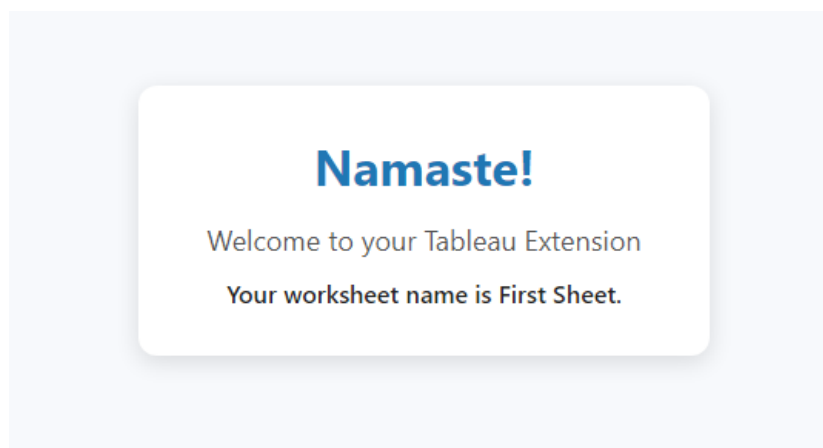
  <!-- Our extension's code -->
  <script src="./Welcome.js"></script>
</head>
<body>
  <h1 id="welcomeMessage">Namaste!</h1>
  <h2 id="content">Your worksheet name...</h2>
</body>
</html>

```

12. Select your extension again to refresh it. You should now see your worksheet name displayed along with the welcome message.



13. As I mentioned earlier, this is your own webpage, so you can customize it using CSS and modify your HTML code to improve its appearance and make it more visually appealing.



14. That's it! You have successfully created your first Tableau Extension. To stop the server, press `Ctrl + C` on cmd.

This is just the beginning. Once you understand the basics, you can start building more advanced extensions. You can even use AI tools like Claude Code, ChatGPT, GitHub Copilot, or any other tool of your choice to speed up the development process.

I recently built a Simple KPI Builder extension, and the experience was a lot of fun. There is so much more you can create once you start exploring what Tableau Extensions can do.

That's it for now. See you in the next one!

Thanks,

Mohit